

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K.Vinay

Roll No.: A24126552086

Date: 21-09-2026

1. TITLE

Development of a Basic Web Server using Express.js

2. AIM

To build and deploy a basic web server using the Express.js framework to handle different HTTP GET routes and serve JSON and HTML responses.

3. OBJECTIVE

The objectives of this experiment are:

- To initialize an Express.js application.
- To define multiple endpoints (routes) such as '/', '/students', and '/about'.
- To send plain text, HTML snippets, and JSON arrays as HTTP responses.
- To start the server listening on a specific network port.

4. THEORY

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies the process of creating server routes, handling HTTP requests, and managing responses (like HTML strings or JSON data) compared to using the native Node.js http module directly.

5. ALGORITHM / PROCEDURE

1. Import the express module and initialize the application.
2. Define a port number (e.g., 3000) for the server to listen on.
3. Create a root route '/' that responds with a welcome message using res.send().
4. Create a '/students' route that returns an array of student names using res.json().
5. Create an '/about' route that serves an HTML formatted string describing the application.
6. Start the server using app.listen() and log a confirmation message to the console.

6. CODE EXPLANATION

The express module is instantiated by calling express(). Different routes are assigned using app.get(). The '/' path sends standard plain-text responses, whereas the '/students' path employs res.json() to convert a standard

JavaScript array into a well-formatted JSON response body. The '/about' path demonstrates returning raw HTML markup. Finally, app.listen() starts the server asynchronously on port 3000.

7. PROGRAM

3_app.js

```
const express = require("express");

const app = express();

const PORT = 3000;

// Home route
app.get("/", (req, res) => {
  res.send("Welcome to Student Server");
});

// Students route
app.get("/students", (req, res) => {

  const students = [
    "Vinay",
    "charan",
    "vijay",
    "srikar",
    "harshad"
  ];

  res.json(students);
});

// About route
app.get("/about", (req, res) => {

  res.send(`
    <h1>About Application</h1>
    <p>This is a simple Express.js Student Server.</p>
    <p>Created using Node.js and Express.js.</p>
  `);
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	GET /	Plain text response	Welcome text sent successfully	Pass

TC-02	GET /students	JSON array	JSON payload is received correctly	Pass
TC-03	GET /about	HTML response	HTML string is rendered securely	Pass

9. OUTPUT

```

13 app.get("/students", (req, res) => {
15   const students = [
19     "srikar",
20     "harshad"
21   ];
22
23   res.json(students);
24 });
25
26 // About route
27 app.get("/about", (req, res) => {
28
29   res.send(`
30     <h1>About Application</h1>
31     <p>This is a simple Express.js Student Server.</p>
32     <p>Created using Node.js and Express.js.</p>
33   `);
34 });
35
36 // Start server
37 app.listen(PORT, () => {
38   console.log(`Server running at http://localhost:${PORT}`);
39 });

```

Problems Output Debug Console Terminal Ports

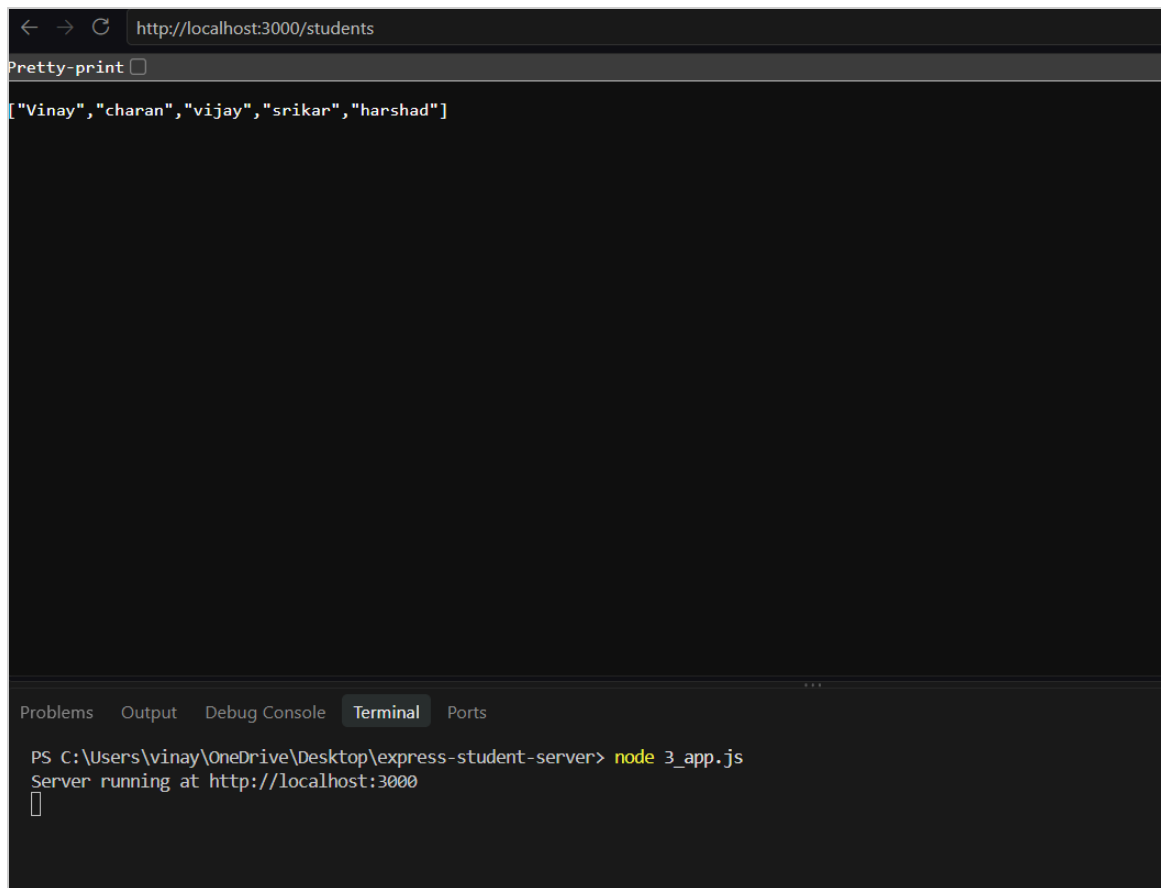
PS C:\Users\vinay\OneDrive\Desktop\express-student-server> node 3_app.js
Server running at http://localhost:3000

← → ↻ http://localhost:3000/

Welcome to Student Server

Problems Output Debug Console Terminal Ports

PS C:\Users\vinay\OneDrive\Desktop\express-student-server> node 3_app.js
Server running at http://localhost:3000



10. RESULT

Thus, a fundamental web server demonstrating routing, HTML responses, and JSON payload delivery was successfully developed utilizing Express.js.